

Lecture Notes: Introduction to React

Course: IT / Web APP Development 2

Topic: Introduction to React — History, Advantages & Disadvantages, File Structures, and Required Technologies

Duration: 90 minutes

1. Learning Objectives

By the end of this lecture, students will be able to:

1. Define React and explain its role in modern web development.
2. Trace the history and evolution of React from its origins at Facebook to its current status.
3. Identify the key advantages and disadvantages of using React.
4. Describe common React project file structures and folder organization strategies.
5. List and explain the required technologies and prerequisites for React development.

2. What is React?

React (also known as React.js or ReactJS) is an **open-source JavaScript library** for building user interfaces, particularly for single-page applications. It is maintained by Meta (formerly Facebook) and a large community of individual developers and companies. React allows developers to create reusable UI components that manage their own state, making it possible to build complex, interactive web applications efficiently.

React is often referred to as a "frontend JavaScript framework," but technically it is a **library** focused on the view layer of an application. It uses a **Virtual DOM** (Document Object Model) to optimize rendering performance — instead of manipulating the browser's DOM directly, React creates a virtual representation of the UI in memory, calculates the most efficient way to update the browser's DOM, and applies only the necessary changes.

3. History of React

3.1 Origins at Facebook (2011)

React was initially created by **Jordan Walke**, a software engineer at Facebook, in **2011**. Walke was inspired by **XHP**, an HTML component library for PHP developed at Facebook, which introduced the concept of composable components.

Before React, Facebook faced a significant challenge: whenever a state change occurred in a web application, the entire page would re-render, causing users to lose previously stored information and resulting in poor performance. Walke started working on a prototype called **FaxJS** to make this process

more efficient while providing a reasonable user experience. This prototype contained several critical characteristics of React, including updating views with minimal DOM manipulation.

React was first used in production for **Facebook's News Feed** feature in 2011.

3.2 Expansion to Instagram (2012)

In **2012**, Facebook acquired Instagram. One of the first products Instagram wanted to build was its website. After examining Facebook's technology stack, **Pete Hunt** made a case to use React on Instagram's website. At the time, React was closely coupled to Facebook's stack. Pete Hunt did most of the work required to decouple React from Facebook's stack so that Instagram could use it, making Instagram the **first external product to use React**.

3.3 Open Source Release (2013)

In **May 2013**, React was **open-sourced** at JSConf in the United States. The initial public release was version **V0.3.0** in **July 2013**.

3.4 Patent Grant and Community Growth (2015)

In **April 2015**, Facebook released a patent regarding the usage of React, which essentially stated: "In as much as you do not sue us for patent infringement, we will give you this software for free; you can sell, distribute, import, make, and use this product." This move encouraged widespread adoption across the developer community.

Since then, React has been actively used in production by companies such as **Trello, Slack, Docker, Airbnb, Khan Academy, The New York Times, and Codecademy**, among many others.

3.5 Timeline Summary

Year	Milestone
2011	Jordan Walke creates FaxJS prototype; React first used in Facebook News Feed
2012	Instagram becomes first external product to use React
2013	React open-sourced at JSConf (May); initial public release V0.3.0 (July)
2015	Facebook releases patent grant
Present	React is one of the most widely used frontend libraries globally

4. Advantages and Disadvantages of React

4.1 Advantages

1. **Better Efficiency via Virtual DOM** — React keeps all app features in a virtual DOM. All changes made in the DOM are automatically updated, resulting in faster rendering and improved application performance.
2. **Component-Based Architecture** — React allows developers to create reusable components. You can easily make segments per your requirements, which promotes code reuse, modularity, and maintainability.
3. **Web Optimization Friendly** — Using React, you can make lightweight web applications. React can run on the server-side, and the virtual DOM is delivered to the browser, improving initial load times.
4. **Large Ecosystem and Community Support** — React benefits from being the most-trained framework in public training corpora. It has a vast ecosystem of libraries, tools, and community resources.
5. **Mobile Development with React Native** — React.js can be used to build different types of apps, from single-page applications (SPAs) to server-side apps and **mobile apps with React Native**.
6. **Backed by Meta (Facebook)** — React's backing by Meta makes it the default choice for complex, long-lived enterprise applications.
7. **Debugging and Developer Tools** — React provides high-performance design tools and browser extensions (React Developer Tools) that make debugging and inspecting component hierarchies easier.

4.2 Disadvantages

1. **Steep Learning Curve** — React can have a steeper learning curve as it often requires understanding additional libraries and patterns, such as state management, routing, and build tooling.
2. **JSX Syntax Can Be Uncomfortable** — Developers who are new to React may find JSX (JavaScript XML) uncomfortable to write. JavaScript expressions must be embedded within curly braces `{}`, and variables and functions cannot be defined directly inside JSX.
3. **Rapid Pace of Development** — ReactJS is constantly experiencing updates in its features and functionality. This rapid pace of development can make it difficult for developers to keep up with best practices.
4. **Complexity and Choice Overload** — The State of JS 2024 survey flagged **excessive complexity** and **choice overload** as common React pain points. Developers must choose their own router, store, and build tooling.
5. **Performance Concerns in Some Scenarios** — Critics argue that despite improvements, React can still be the slowest frontend framework in certain scenarios, with memoization and re-rendering issues that require careful optimization.
6. **Hooks Rules and Constraints** — React's documentation specifies that hooks can only be called at the top of components, never inside loops or conditions. `useEffect` requires a dependency array covering every reactive value it reads, which can be error-prone.
7. **Ecosystem Fragmentation** — React and its ecosystem have been described as "complex and fragmented," with ongoing debates about the best practices and the role of frameworks like Next.js.

5. File Structures in React

React does not have an opinionated way to structure projects. However, there are several common approaches popular in the ecosystem.

5.1 Grouping by Features or Routes

One common way to structure projects is to **locate CSS, JS, and tests together** inside folders grouped by feature or route.

```
common/  
  Avatar.js  
  Avatar.css  
  APIUtils.js  
  APIUtils.test.js  
feed/  
  index.js  
  Feed.js  
  Feed.css  
  FeedStory.js  
  FeedStory.test.js  
  FeedAPI.js  
profile/  
  index.js  
  Profile.js  
  ProfileHeader.js  
  ProfileHeader.css  
  ProfileAPI.js
```

The definition of a "feature" is not universal — it is up to you to choose the granularity. If you cannot come up with a list of top-level folders, ask the users of your product what major parts it consists of and use their mental model as a blueprint.

5.2 Grouping by File Type

Another popular way to structure projects is to **group similar files together**:

```
api/  
  APIUtils.js  
  APIUtils.test.js  
  ProfileAPI.js  
  UserAPI.js  
components/  
  Avatar.js  
  Avatar.css  
  Feed.js  
  Feed.css  
  FeedStory.js  
  FeedStory.test.js  
  Profile.js  
  ProfileHeader.js
```

ProfileHeader.css

Some developers prefer to go further and separate components into different folders depending on their role in the application. For example, **Atomic Design** is a design methodology built on this principle.

5.3 Typical Create React App Structure

When you create a new React project using `create-react-app`, the default structure looks like this:

```
my-app/
├── node_modules/
├── public/
│   ├── index.html
│   ├── favicon.ico
│   └── manifest.json
├── src/
│   ├── App.js
│   ├── App.css
│   ├── App.test.js
│   ├── index.js
│   ├── index.css
│   ├── logo.svg
│   └── reportWebVitals.js
├── .gitignore
├── package.json
├── package-lock.json
└── README.md
```

Key files:

- `public/index.html` — the main HTML file where the React app is mounted.
- `src/index.js` — the JavaScript entry point that renders the root component.
- `src/App.js` — the main application component.
- `package.json` — lists dependencies and scripts (start, build, test, eject).

5.4 Recommended Folder Structure for Scalable Projects

For larger projects, a professional structure may include:

```
src/
├── assets/           # Static assets (images, icons, fonts)
├── components/      # Reusable UI components
│   ├── common/     # Common components (Button, Input, etc.)
│   └── forms/       # Form-specific components
├── hooks/           # Custom React hooks
├── pages/           # Page components (one per route)
├── services/        # API services and utilities
├── store/           # State management (Redux, Context, etc.)
├── utils/           # Helper functions
└── App.js           # Main application component
```

5.5 Best Practices for File Structure

- **Avoid too much nesting** — Limit yourself to a maximum of three or four nested folders within a single project. Deep nesting makes relative imports harder to write and update.
- **Don't overthink it** — If you are just starting a project, don't spend more than five minutes on choosing a file structure. Pick any approach and start writing code. You will likely want to rethink it after writing some real code.
- **Colocation** — Keep files that often change together close to each other. This principle is called "colocation."
- **Start simple** — Begin by keeping all files in a single folder. Eventually it will grow large enough that you will want to separate some files.

6. Required Technologies and Prerequisites

6.1 Core Prerequisites

Before learning React, students should have a solid foundation in:

Prerequisite	Description
HTML	Semantic markup, forms, and document structure
CSS	Layout (Flexbox, Grid), responsive design
JavaScript (ES6+)	Arrow functions, modules, destructuring, spread/rest, classes, Promises, async/await, array methods (<code>map</code> , <code>filter</code> , <code>reduce</code>)
Git and GitHub	Basic version control (branching, pull requests, resolving conflicts)
CLI Comfort	Navigating the terminal/command line

Note: The DOM (Document Object Model) is covered in a separate discussion and is not included as a prerequisite here.

6.2 Required Technologies and Tools

Technology	Purpose
------------	---------

Node.js	JavaScript runtime environment required for local React development. Most JavaScript tooling you'll want to use for development requires Node.js.
npm (Node Package Manager)	Comes bundled with Node.js. Used to install React and other dependencies.
JSX	JavaScript syntax extension that lets you write markup-like code directly inside JavaScript. It compiles to lightweight function calls.
Babel	A JavaScript compiler/transpiler that converts JSX syntax and modern JavaScript (ES6+) into backward-compatible JavaScript that browsers can understand. Babel transforms JSX elements into <code>React.createElement</code> calls.
Webpack	A module bundler that organizes and bundles JavaScript files, React code, and other assets into a single bundle for the browser.
Code Editor (VS Code)	Recommended editor for React development. Install extensions like ES7+ React/Redux snippets, ESLint, and Prettier.
Web Browser (Chrome)	For viewing and debugging React applications. React Developer Tools extension is highly recommended.

6.3 What `create-react-app` Includes

When you run `npm create-react-app my-app`, the tool sets up a development environment that includes:

- **Webpack** — for bundling modules
- **Babel** — for transpiling JSX and modern JavaScript
- **ESLint** — for code linting and error checking

6.4 Optional but Recommended

Technology	Purpose
React Router	For client-side routing in single-page applications

State Management (Redux, Zustand, Context API)	For managing complex application state
TypeScript	For type-safe React development
Testing Library (Jest, React Testing Library)	For unit and integration testing
Vite	A modern, faster alternative to Webpack for React project bundling

7. Summary

- React is a **JavaScript library** created by **Jordan Walke** at Facebook in **2011** for building user interfaces.
- It was first used in Facebook's News Feed, expanded to Instagram in 2012, and was **open-sourced in 2013**.
- **Advantages** include the Virtual DOM, component-based architecture, large ecosystem, and mobile development with React Native.
- **Disadvantages** include a steep learning curve, JSX syntax challenges, rapid development pace, and ecosystem complexity.
- React projects can be structured by **features/routes** or by **file type**; there is no single "correct" structure, but colocation and avoiding deep nesting are recommended.
- **Required technologies** include Node.js, npm, JSX, Babel, Webpack, and a code editor like VS Code.
- **Prerequisites** include solid HTML, CSS, JavaScript (ES6+), Git basics, and CLI comfort. (*The DOM is covered separately.*)

8. Review Questions

1. Who created React and in what year?
2. What was the name of the prototype that eventually became React?
3. What is the Virtual DOM and why is it important?
4. List three advantages and three disadvantages of using React.
5. Describe two common approaches to structuring a React project.
6. What is JSX and what tool is used to transpile it?
7. What is the role of Node.js in React development?
8. What does **create-react-app** include out of the box?

9. References

- [W3Schools / Coursera — React Introduction](#)
- [Pusher Blog — The Evolution of React](#)
- [Educative — Node.js vs React.js](#)
- [React Documentation — File Structure](#)
- [Coursera — React Learning Roadmap \(2026\)](#)
- [BrowserStack — Vue vs React \(2025\)](#)
- [Vercel Docs — Vue vs React \(2026\)](#)
- [Babel Documentation](#)
- [NetGuru — Professional React Project Structure \(2025\)](#)

Instructor's Note: Encourage students to set up their development environment before the next laboratory session. The next lab activity will focus on installing Node.js, VS Code, and creating their first React application using `create-react-app`.